

Reviewer Pack — Matroid Rank Deficit in Monotone DNF Depth for k-CLIQUE

Entry 86fc235d2aba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 13:48:08 UTC

Matroid Rank Deficit in Monotone DNF Depth for k-CLIQUE

Entry ID: 86fc235d2aba **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 13:48:08 UTC

1. Conjecture

Field A (mathematical branch): MATROID_THEORY **Field B** (complexity object): MONOTONE_CIRCUIT_COMPLEXITY

Statement:

Let M be a matroid of rank r on n elements, and let $\mu(M) = r - \text{rank}(\text{groundset} - \text{closure}(\text{DNF_terms}))$. For any monotone DNF formula with size $S(n)$, $\mu(M) \leq \log n$. For the k -CLIQUE indicator function, $\mu(M) \geq \Omega(n^{1 - 1/(k-1)})$.

Rationale (proposer's reasoning):

Matroid rank functions are submodular and capture combinatorial dependencies. By linking matroid rank deficits to DNF size, we create a structural invariant that could amplify communication complexity lower bounds for k -CLIQUE via combinatorial duality. This avoids algebraic barriers by staying in discrete matroid theory.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 041dec4c32820998

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    result = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented = [row + [b[i]] for i, row in enumerate(A)]
    for i in range(n):
        max_row = max(range(i, m), key=lambda r: abs(augmented[r][i]))
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]
        for j in range(i+1, m):
            factor = augmented[j][i] / augmented[i][i]
            for k in range(n + 1):
                augmented[j][k] -= factor * augmented[i][k]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (augmented[i][-1] - sum(augmented[i][j] * x[j] for j in range(i+1, n))) / augmented[i][i]
    return x

def rank(A):
    m, n = len(A), len(A[0])
    A_rref = gaussian_elimination(A, [0]*n)
    rank = 0
    for row in A_rref:
```

```

        if any(row[j] != 0 for j in range(n)):
            rank += 1
    return rank

def closure(DNF_terms, groundset):
    closure_set = set(groundset)
    changed = True
    while changed:
        changed = False
        for term in DNF_terms:
            if all(x in closure_set for x in term):
                for x in term:
                    if x not in closure_set:
                        closure_set.add(x)
                        changed = True
    return closure_set

def matroid_rank_deficit(M, DNF_terms):
    groundset = set(range(len(M)))
    closure_set = closure(DNF_terms, groundset)
    rank_M = rank(M)
    rank_closure_diff = rank_M - rank(groundset - closure_set)
    return rank_closure_diff

def generate_k_clique_dnf(n, k):
    edges = list(combinations(range(n), 2))
    clique_edges = random.sample(edges, k*(k-1)//2)
    DNF_terms = []
    for i in range(k):
        term = set()
        for j in range(i+1, k):
            term.add(clique_edges[i*k + j - (i+1)*(i//2)][0])
            term.add(clique_edges[i*k + j - (i+1)*(i//2)][1])
        DNF_terms.append(term)
    return DNF_terms

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5):
            M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
            DNF_terms = generate_k_clique_dnf(n, k=3)
            mu_M = matroid_rank_deficit(M, DNF_terms)
            total_metric_value += mu_M
            instances_tested += 1

    if instances_tested > 0:
        mean_metric_value = total_metric_value / instances_tested
        support_fraction = sum(1 for n in n_values for _ in range(5) if matroid_rank_deficit(M, DNF_terms) == 0) / (len(n_values) * 5)
    else:

```

```

    mean_metric_value = None
    support_fraction = 0

    return {
        "metric_name": "mu_M",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": counterexample if not conjecture_holds else ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["instances_tested"] > 0) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no seeds tested")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b53c5454.py", line 126, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b53c5454.py", line 101, in run_trial
    DNF_terms = generate_k_clique_dnf(n, k=3)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b53c5454.py", line 85, in generate_k_clique_dnf
    term.add(clique_edges[i*k + j - (i+1)*(i//2)][0])
              ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError during DNF term generation for k=3, preventing result collection. | next: Debug generate_k_clique_dnf() implementation for k=3 edge indexing

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	43335
2	preregistration	ollama_remote	qwen3:8b	0	0	24246
3	novelty	ollama_remote	qwen3:8b	0	0	21202
4	novelty	ollama_remote	qwen3:8b	0	0	16701
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18959
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14318
7	judge	ollama_remote	qwen3:8b	0	0	19673

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 158434 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/86fc235d2aba.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/86fc235d2aba.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/86fc235d2aba.tar.gz (if generated)